

Computer Vision (COL780) Assignment 3

Object Detection and Recognition

PARTH PATEL (2021CS10550)

April 3, 2024

§1 Tools and technology used

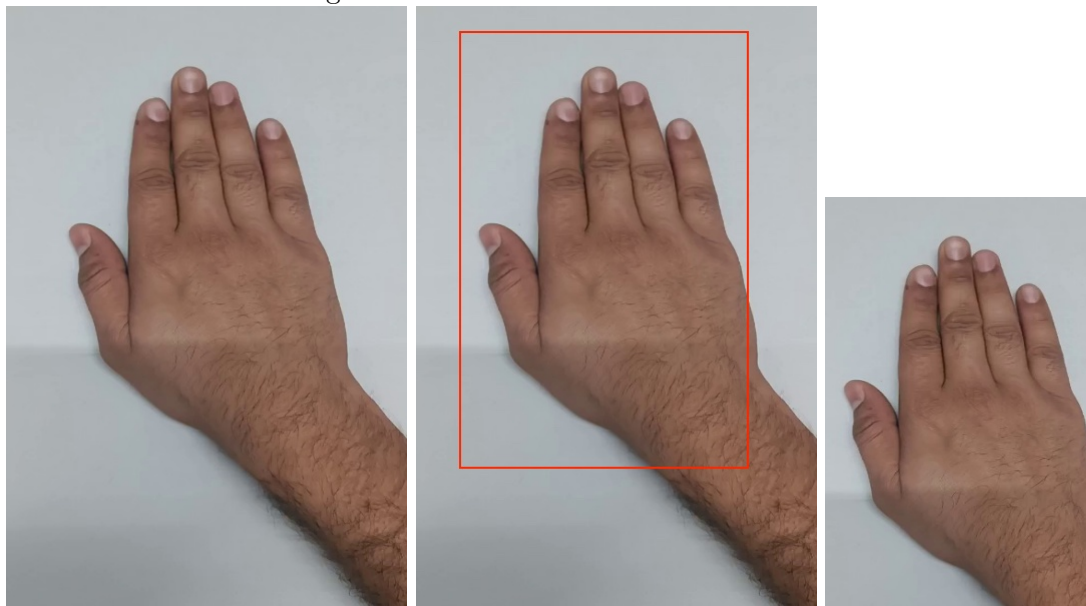
I used the following python packages, libraries, modules and frameworks - opencv, sklearn, random, numpy, mediapipe, pickle, os, sys.

- **OpenCV (Open Source Computer Vision Library):** OpenCV is a popular open-source computer vision and machine learning software library. It provides various tools and functions to perform real-time computer vision tasks such as object detection, image processing, feature detection, and more. OpenCV is widely used in applications such as robotics, augmented reality, facial recognition, and gesture recognition.
- **scikit-learn (sklearn):** Scikit-learn is a versatile machine learning library for Python. It provides simple and efficient tools for data mining and data analysis, built on top of other scientific computing libraries such as NumPy, SciPy, and matplotlib. Scikit-learn includes various algorithms for classification, regression, clustering, dimensionality reduction, model selection, and preprocessing. We will use SVM.
- **random:** The random module is a part of Python's standard library and provides functions for generating random numbers. It is commonly used for tasks such as shuffling sequences, generating random samples, and simulating random events.
- **NumPy:** NumPy is a fundamental package for scientific computing with Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays efficiently. NumPy is often used in combination with other libraries for tasks involving numerical computations and data manipulation.
- **MediaPipe:** MediaPipe is an open-source framework developed by Google that offers solutions for building scalable, real-time machine learning pipelines for various tasks such as object detection, pose estimation, hand tracking, and facial recognition. It provides pre-trained models and easy-to-use APIs for incorporating machine learning into multimedia applications.
- **pickle:** Pickle is a module in Python's standard library used for serializing and deserializing Python objects. It enables the conversion of Python objects into a byte stream, which can then be saved to a file or transmitted over a network. Pickle is commonly used for saving trained machine learning models, configuration settings, and other complex data structures.

- **os (Operating System Interface):** The `os` module provides a wide range of functions for interacting with the operating system, such as managing files and directories, accessing environment variables, executing system commands, and more. It abstracts away the differences between different operating systems, allowing Python code to be written in a platform-independent manner.
- **sys:** The `sys` module provides access to some variables used or maintained by the Python interpreter and to functions that interact strongly with the interpreter. It typically deals with system-specific parameters and functions. In the context of your usage, it might be used for handling command-line arguments, accessing system-specific parameters, or managing the Python runtime environment.

§2 Hand Detection

We will make use of `mediapipe` to detect the hands and then crop the hand from training dataset images. When we run `ROI_coordinates.py`, the program detects the hands in the training dataset images and generate a directory containing the the cropped images which we will be used later. In case the bounding box (scale = 1.3 of detected region) coordinates exceed the limits of the image itself, we set the pixels colour of the bounding box which are not in image to black.



§3 Preprocessing and computing descriptor

§3.1 Pre-processing

We convert the cropped hand images that we obtained in the previous step to grayscale and then resize them to size 64x128.

§3.2 Computing HoG descriptors

Now we compute the HoG descriptor for all the 64x128 sized grayscale hand images. The HoG descriptor is a 3780 feature length vector. We use a slightly modified version of HoG descriptor than what was proposed in the original paper. We use weighted voting. The steps to compute the HoG descriptor is -

1. Take the image derivatives and compute its gradient and magnitude.
2. Divide the image into 128 blocks (of size 8x8).
3. For each 8x8 block compute a vector (histogram) with number of bins as 9. For each pixel gradient, depending upon the bin it belongs to, poll the corresponding bins with the magnitude of that pixel. (we will use weighted voting)
4. Combine 4 8x8 blocks to compute a block descriptor of length $4 \times 9 = 36$. Normalize it by dividing each element by its magnitude. This normalization ensures the descriptor is illumination invariant.
5. Since there are $(8 - 1) \cdot (16 - 1) = 105$ such 36-length block descriptors, the total length of the feature vector is $105 \cdot 36 = 3780$.

§3.3 Challenges encountered

Initially I was using the opencv HoG descriptor and that was giving me good results (classifying **my** hands correctly). Then when I implemented my own HoG descriptor, the SVM always classified my hands as open. It took me a while to figure out what is the difference. I came to know that the opencv HoG descriptor uses weighted polling while computing the histogram. To elaborate, we increment both the neighbouring bins (given the gradient of that pixel) by some fraction of the magnitude (the fractions are determined from the facts that how far is the given gradient from the neighbouring bin range centres). This slight modification greatly improved the performance of my real-life application.

§4 Hand Pose Recognition - Training

Once we compute the feature vector for all of the training dataset, we will use SVM (support vector machine) to train our classifier.

§4.1 Challenges encountered

The data can be biased with having more images of a one hand (say, left) than the other (say, right). To overcome this bias, I flipped a given image with probability of 0.5. This ensures that the final dataset on which the model has been trained contains almost equal number of "left" and "right" hand images in the limit of large training dataset size. This also greatly improved the performance of my real-life application.

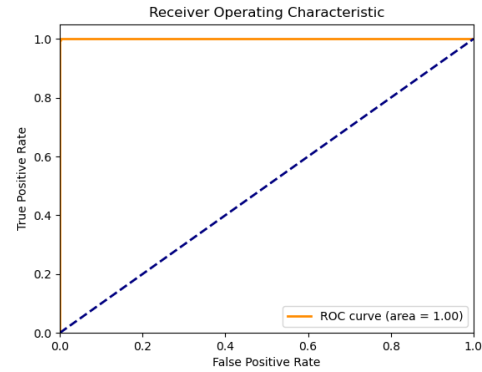
§5 Testing

I used the validation dataset for testing. We first get the cropped hand(s) by using mediapipe and then classify the gesture with our trained classifier. Model's task is to distinguish between a "closed hand" (positive) and an "open hand" (negative)

```
print("False Positive: ", false_positive)
print("False Negative: ", false_negative)
print("True Positive: ", true_positive)
print("True Negative: ", true_negative)
print("Accuracy: ", (true_positive + true_negative) / (true_positive + true_negative + false_positive + false_negative))
print("Precision: ", true_positive / (true_positive + false_positive))
print("Recall: ", true_positive / (true_positive + false_negative))
print("F1 Score: ", 2 * true_positive / (2 * true_positive + false_positive + false_negative))
print("Specificity: ", true_negative / (true_negative + false_positive))
```

Metric	Value
False Positive	2
False Negative	0
True Positive	330
True Negative	640
Accuracy	0.99794
Precision	0.99398
Recall	1.0000
F1 Score	0.99698
Specificity	0.99688
AUROC	0.99999

(a) Metrics



(b) ROC Curve

Figure 1: Evaluation Metrics and ROC Curve

As we can see, the model works quite well when tested with the validation data set. We are getting two false positives (classified as close but were open). The images are the following (bottom hands of the two different raw images are classified as closed) -

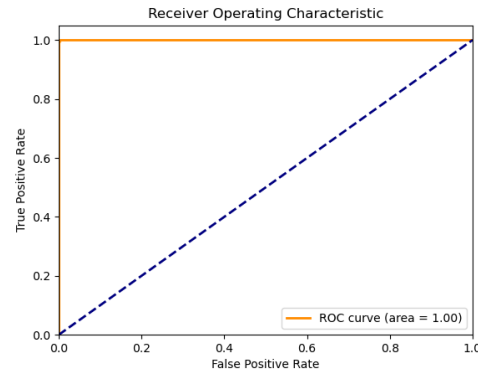


We can clearly see that this is not a clear image and can be considered a boundary case.

If we consider Model's task is to distinguish between a "closed hand" (negative) and an "open hand" (positive) then the metrics and ROC curve is different than what was obtained previously.

Metric	Value
False Positive	0
False Negative	2
True Positive	640
True Negative	330
Accuracy	0.99794
Precision	1.0
Recall	0.99688
F1 Score	0.99844
Specificity	1.0
AUROC	0.99999

(a) Metrics



(b) ROC Curve

Figure 2: Evaluation Metrics and ROC Curve

§6 Hand Gesture-Controlled Photography Application: A Hands-On Approach

I have implemented an application which takes a photo based on hand-gestures. Its key features and functionality is as follows -

- A photo will be taken (and saved) 5 seconds after a person shows at least one open hand. This ensures that the person gets enough time to pose for the photograph.
- The application closes when the person shows only closed hand(s) (i.e. no open hands).
- During the time during of 5 seconds after the gesture of taking photo is made, even if closed hand is shown, the application would not be closed.
- The application can be closed only when the application is in 'standby' mode.
- There is no restriction on the type of pose you want to make while the photo is getting captured.
- Note that even if the frame contains at least one open hand (doesn't matter how many closed hands are there), an image will be taken 5 second after that gesture. To close the application, the frame needs to have all (and at least one) of the detected hands as closed.

§7 References and Acknowledgement

Analytics Vidhya : HoG feature descriptor

Stack Overflow : OpenCV HOGDescriptor Python (Second answer)

Towards Data Science : A Look at Precision, Recall, and F1-Score